

Luồng 2: Chat — Authenticated & Anonymous

Mô tả chi tiết pipeline chat AI, bao gồm luồng authenticated (Firebase) và anonymous (API Key).

2.1 Tổng quan

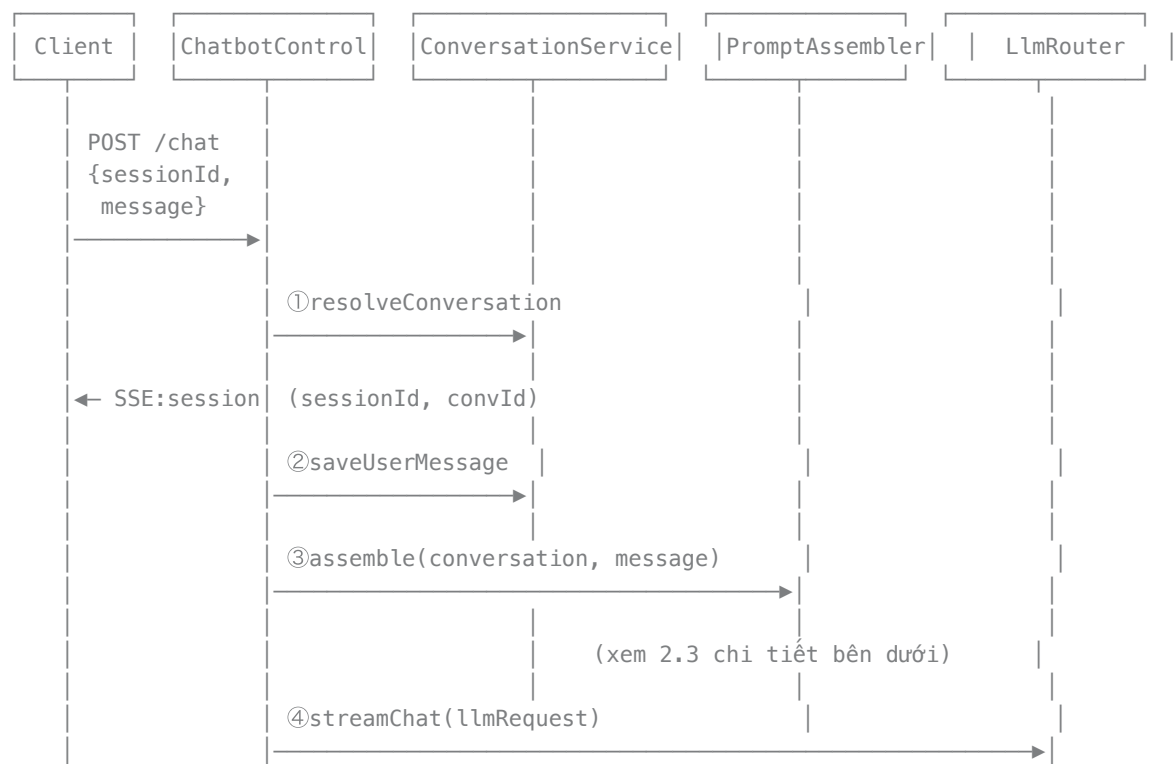
Hệ thống cung cấp 2 endpoint chat:

Endpoint	Auth	User	Giới hạn
POST /api/v1/chatbot/chat	Firebase ID Token	Authenticated user	Không giới hạn
POST /api/v1/public/chat	X-API-Key header	Anonymous	10 tin nhắn/session

Cả hai đều trả về **SSE (Server-Sent Events)** streaming response.

2.2 Chat Pipeline — Authenticated User

Sequence Diagram





Chi tiết từng Step

Step ① — Session Resolution

File: service/ConversationService.java → resolveConversation()

```

public Conversation resolveConversation(String userId, String username, UUID sessionId) {
    if (sessionId != null) {
        // Case A: Có sessionId + conversation ACTIVE → resume
        Optional<Conversation> active = findActiveBySessionId(sessionId);
        if (active.isPresent()) return active.get();

        // Case B: Có sessionId nhưng conversation DONE → tạo mới với sessionId mới
        Optional<Conversation> existing = findBySessionId(sessionId);
        if (existing.isPresent()) return createConversation(userId, username, null);

        // Case C: sessionId chưa tồn tại → tạo mới với sessionId đó
        return createConversation(userId, username, sessionId);
    }
    // Case D: Không có sessionId → tạo mới hoàn toàn
    return createConversation(userId, username, null);
}

```

Conversation mới được tạo với:

- sessionId : UUID (client gửi hoặc auto-generate)
- status : ACTIVE
- type : BOT
- doctor : "BOT"
- promptName : "medical_general"

Step ② — Save User Message

File: service/MessageService.java → saveUserMessage()

```

Message message = Message.builder()
    .conversation(conversation)
    .senderId(userId) // Firebase UID
    .senderRole(SenderRole.USER)

```

```

        .content(content)
        .contentType(ContentType.TEXT)
        .build();
messageRepository.save(message);

```

Step ③ — Prompt Assembly (chi tiết ở section 2.3)

Step ④ — LLM Streaming

File: service/llm/OpenAiClient.java → streamChat()

```

// 1. Convert LlmRequest → OpenAI message format
List<ChatMessage> messages = promptAssembler.toOpenAiMessages(request);

// 2. Build request body
Map<String, Object> requestBody = Map.of(
    "model", request.getModel(), // "gpt-4o"
    "temperature", request.getTemperature(), // 0.3
    "messages", messageList,
    "stream", true
);

// 3. POST /chat/completions với SSE
webClient.post()
    .uri("/chat/completions")
    .bodyValue(requestBody)
    .accept(MediaType.TEXT_EVENT_STREAM)
    .retrieve()
    .bodyToFlux(String.class)
    .filter(data -> !data.equals("[DONE]"))
    .mapNotNull(this::extractContent); // parse delta.content

```

SSE chunk format từ OpenAI:

```

{"choices":[{"delta":{"content":"Xin"}}]}
{"choices":[{"delta":{"content":" chào"}}]}
{"choices":[{"delta":{"content":"!"}}]}
[DONE]

```

LlmRouter route request dựa trên model name:

- gpt-4o , gpt-4o-mini , gpt-4-turbo , o1 , o3-mini → OpenAiClient
- Có thể mở rộng thêm provider (Gemini, etc.)

Step ⑤ — Save Bot Response

Sau khi stream hoàn tất, fullResponse (StringBuilder) chứa toàn bộ text → lưu DB:

```

Message botMessage = messageService.saveBotMessage(conversation, fullResponse.toString());

```

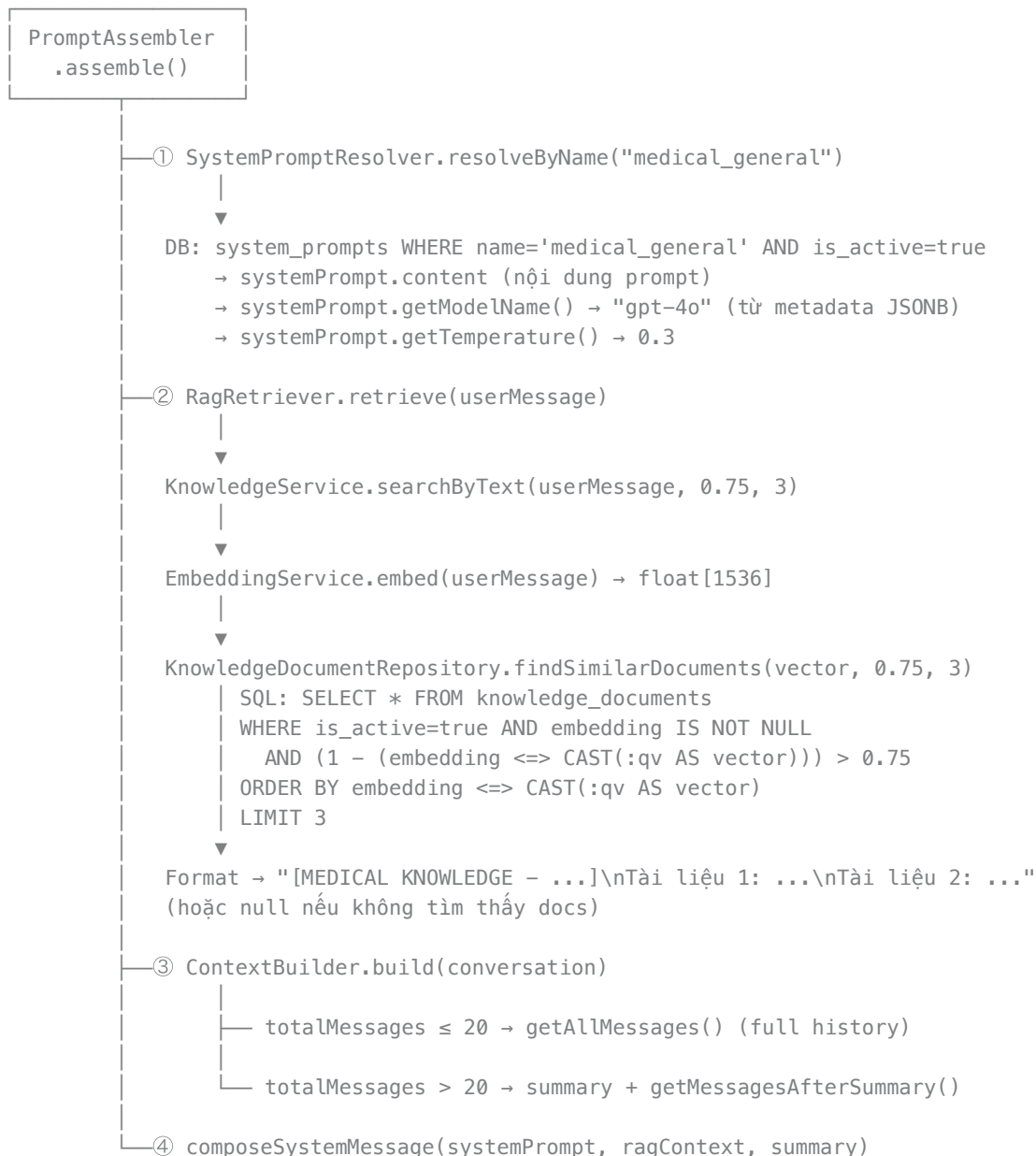
Step ⑥ — Auto Summary Check

File: service/prompt/SummaryGenerator.java → needsSummary()

```
public boolean needsSummary(Conversation conversation) {
    long totalMessages = messageService.countByConversationId(conversation.getId());
    if (totalMessages <= 20) return false;           // Dưới ngưỡng
    if (conversation.getSummary() == null) return true; // Chưa có summary
    // Đã có summary → check số tin nhắn mới > 20
    List<Message> newMsgs = messageService.getMessagesAfterSummary(...);
    return newMsgs.size() > 20;
}
```

Nếu cần summary → gọi generateSummaryAsync() (chạy @Async, không block response).

2.3 Prompt Assembly Pipeline — Chi tiết



↓
▼
Final System Prompt = Base Prompt
+ "\n\n" + [MEDICAL KNOWLEDGE]
+ "\n\n" + [TÓM TẮT CUỘC HỘI THOẠI TRƯỚC ĐÓ]

Message format gửi cho OpenAI

```
{
  "model": "gpt-4o",
  "temperature": 0.3,
  "stream": true,
  "messages": [
    {"role": "system", "content": "<systemPrompt + RAG + summary>"},
    {"role": "user", "content": "tin nhắn cũ 1"},
    {"role": "assistant", "content": "trả lời cũ 1"},
    {"role": "user", "content": "tin nhắn mới nhất"}
  ]
}
```

2.4 Chat Pipeline — Anonymous User

Khác biệt so với Authenticated

Aspect	Authenticated	Anonymous
Auth	Firebase ID Token	X-API-Key header
Filter	FirebaseAuthFilter	PublicApiKeyFilter
Endpoint	/api/v1/chatbot/chat	/api/v1/public/chat
userId	Firebase UID	"ANONYMOUS"
username	Firebase displayName	"Guest"
Conv type	BOT	ANONYMOUS
Giới hạn	Không	10 messages/session
Summary	Có (async)	Không
Cleanup	Không	Hard-delete sau 24h

Sequence Diagram — Anonymous





Step ① — Resolve Anonymous Conversation

File: service/ConversationService.java → resolveAnonymousConversation()

```

public Conversation resolveAnonymousConversation(UUID sessionId) {
    if (sessionId != null) {
        Optional<Conversation> active = findActiveBySessionId(sessionId);
        if (active.isPresent()) {
            Conversation conv = active.get();
            // Phải là ANONYMOUS + chưa quá 24h
            if (conv.getType() == ConversationType.ANONYMOUS
                && conv.getCreatedAt().isAfter(LocalDate.now().minusHours(24))) {
                return conv; // Resume
            }
        }
    }
    return createAnonymousConversation(sessionId); // Tạo mới
}

```

Step ② — Message Limit Check

```

long userMessageCount = conversationService.countUserMessages(conversation.getId());
if (userMessageCount >= MAX_MESSAGES_PER_SESSION) { // MAX = 10
    emitter.send(SseEmitter.event()
        .name("limit_reached")
        .data("{\"error\": \"Message limit reached\", \"suggestion\": \"register\"}"));
    emitter.complete();
    return;
}

```

Anonymous Cleanup Job

File: service/AnonymousCleanupJob.java

```
@Scheduled(fixedRate = 3600000) // Chạy mỗi 1 giờ
@Transactional
public void cleanupExpiredAnonymousConversations() {
    LocalDateTime cutoff = LocalDateTime.now().minusHours(24);
    // Tìm anonymous conversations > 24h
    List<Conversation> expired = conversationRepository
        .findByTypeAndCreatedAtBefore(ConversationType.ANONYMOUS, cutoff);
    // Xóa messages trước (FK constraint), rồi xóa conversations
    for (Conversation conv : expired) {
        messageRepository.deleteByConversationId(conv.getId());
    }
    conversationRepository.deleteByTypeAndCreatedAtBefore(ConversationType.ANONYMOUS, cutoff);
}
```

2.5 SSE Events Reference

Event Name	Thời điểm	Data
session	Đầu tiên	{"sessionId":"<uuid>", "conversationId":<long>}
message	Streaming	Text chunk từ LLM (mỗi vài ký tự)
done	Kết thúc	{"messageId":<long>}
limit_reached	Anonymous vượt 10 msg	{"error":"Message limit reached", "suggestion":"register"}
error	Lỗi pipeline	{"error":"<message>"}

2.6 Auto Summary — Chi tiết

Khi nào tạo summary?

- Tổng messages > 20 VÀ chưa có summary → tạo lần đầu
- Tổng messages mới kể từ summary cuối > 20 → regenerate

Summary Generation

File: service/prompt/SummaryGenerator.java

```
Lần đầu:
Input:  M1, M2, ..., M(n-10)  (bỏ 10 tin nhắn gần nhất)
```

Output: Structured summary

Regenerate:

Input: Old Summary + New Messages (trừ 10 gần nhất)

Output: Merged summary

System Prompt cho summary:

- Model: gpt-4o-mini (rẻ hơn, đủ cho summarization)
- Temperature: 0.1
- Output format có cấu trúc: Triệu chứng, Thuốc, Chỉ số y tế, Tiền sử, Diễn biến, Khuyến nghị

Summary được lưu vào:

- `conversation.summary` — nội dung tóm tắt
- `conversation.summary_until_id` — ID message cuối cùng được tóm tắt

2.7 Escalation — Chuyển sang Bác sĩ

